

ANÁLISIS DE COMPORTAMIENTO DE CLIENTES

Retail 2020–2021

Documentación técnica del proyecto
ETL en Python (Pandas) + Cuadro de mando en Power BI

Proyecto de portfolio · Análisis de Datos

1. Contexto y objetivo

Una tienda retail dispone de un histórico de pedidos correspondientes a los ejercicios 2020 y 2021 y desea entender el comportamiento de sus clientes en función de su año de adquisición y de cómo han evolucionado sus hábitos de compra a lo largo del tiempo.

El objetivo de este proyecto es construir un flujo completo de análisis de datos: desde la ingesta y limpieza de un dataset en bruto en formato JSON, pasando por el cálculo de las métricas de negocio solicitadas, hasta la construcción de un cuadro de mando interactivo en Power BI que permita a negocio explorar los resultados.

1.1 Requisitos del enunciado

Se solicita construir un cuadro de mando que permita desglosar la información al menos por:

- Año y mes de adquisición del cliente.
- Estado / provincia del cliente.

Y que muestre, tanto en formato gráfico como en tabla resumen, las siguientes métricas clave:

- % de clientes recurrentes.
- Importe de primeros pedidos.
- Importe de pedidos recurrentes.
- Tiempo medio entre recurrencias.

2. Dataset de partida

El dataset se entrega en formato JSON (una línea por pedido) e incluye información del cliente, de la cabecera del pedido y de las líneas de producto asociadas a cada pedido. Los campos originales son los siguientes:

Campo	Descripción
order_id	Identificador del pedido.
order_date	Fecha del pedido.
first_name / last_name	Nombre y apellidos del cliente.
gender	Género del cliente.
email	Correo electrónico del cliente (usado como identificador único de cliente).
phone	Teléfono del cliente.
state / zip	Estado/provincia y código postal del cliente.
order_items	Lista de líneas de pedido, cada una con sku, qty_ordered, price, discount_amount y product_category.

Al no existir un campo explícito de "cliente", se ha tomado el correo electrónico (email) como clave de identificación de cliente para todo el análisis, al ser el campo más estable y menos propenso a duplicidades por variaciones de escritura (frente al nombre y apellidos).

3. Tratamiento de los datos (ETL en Python)

Todo el proceso de carga, limpieza y transformación se ha realizado en Python con la librería Pandas, en el notebook script_Retail.ipynb. A continuación se detallan los pasos seguidos.

3.1 Carga del dataset

El fichero JSON se carga directamente con Pandas, indicando que cada línea del fichero es un registro independiente:

```
import pandas as pd

df = pd.read_json("dataset.json", lines=True)
```

3.2 Desanidado de las líneas de pedido

Cada pedido contiene una lista de líneas (order_items) con la información de los productos comprados. Para poder trabajar a nivel de línea de producto, se "explota" esa lista (una fila por línea de pedido) y se normalizan sus diccionarios en columnas independientes, que después se concatenan con el resto de campos del pedido:

```
df = df.explode("order_items")

# Convierte cada diccionario en columnas
items = pd.json_normalize(df["order_items"])

# Une las nuevas columnas con el resto del DataFrame
df = pd.concat([df.drop(columns=["order_items"]).reset_index(drop=True),
               items.reset_index(drop=True)],
              axis=1)
```

3.3 Normalización de nombres de columnas

Se renombran las columnas originales (en inglés) a nombres en castellano, más claros para el análisis de negocio:

```
df = df.rename(columns={
    "email": "correo",
    "first_name": "nombre",
    "gender": "genero",
    "last_name": "apellidos",
    "order_date": "fecha_pedido",
    "order_id": "id_pedido",
    "phone": "telefono",
    "state": "estado",
    "zip": "codigo_postal",
    "discount_amount": "descuento",
    "price": "precio",
    "product_category": "categoria_producto",
    "qty_ordered": "cantidad",
    "sku": "sku"
})
```

3.4 Parseo y tipado de los campos

Los campos numéricos y de fecha llegan como texto desde el JSON, por lo que se convierten a los tipos correspondientes para poder operar con ellos:

```
df["fecha_pedido"] = pd.to_datetime(df["fecha_pedido"], format="%d-%m-%Y")
df["codigo_postal"] = df["codigo_postal"].astype(str)
df["descuento"] = df["descuento"].astype(float)
df["precio"] = df["precio"].astype(float)
df["cantidad"] = df["cantidad"].astype(int)
```

Tras el parseo se valida con `df.info()` que los tipos son correctos y con `df[["descuento","precio","cantidad"]].describe()` que las variables numéricas no presentan valores anómalos (por ejemplo, importes o cantidades negativas).

3.5 Cálculo de campos derivados

Se añade el importe de cada línea de pedido, resultado de multiplicar el precio unitario (ya con el descuento aplicado) por la cantidad:

```
df["importe"] = df["precio"] * df["cantidad"]
```

4. Cálculo de las métricas de negocio

A partir del dataset ya limpio y a nivel de línea de pedido, se construyen dos tablas agregadas — pedidos y clientes — sobre las que se calculan las métricas solicitadas en el enunciado.

4.1 Tabla de pedidos

Se agrupan las líneas por `id_pedido` para obtener una fila por pedido, con el correo y estado del cliente (usando el primer valor, ya que son constantes dentro de un mismo pedido) y el importe total del pedido (suma de las líneas). A continuación se ordenan los pedidos de cada cliente por fecha y se numera cada pedido (1º, 2º, 3º...) para poder distinguir el primer pedido de los recurrentes:

```
pedidos = (df.groupby("id_pedido")
            .agg(correo=("correo", "first"),
                 fecha_pedido=("fecha_pedido", "min"),
                 estado=("estado", "first"),
                 importe_pedido=("importe", "sum"))
            .reset_index())

pedidos = pedidos.sort_values(["correo", "fecha_pedido"])
pedidos["num_pedido_cliente"] = pedidos.groupby("correo").cumcount() + 1
```

4.2 Tabla de clientes

A partir de la tabla de pedidos se construye una fila por cliente (correo) con el número total de pedidos, la fecha de adquisición (fecha del primer pedido), el estado y el mes/año de adquisición. Un cliente se considera recurrente cuando ha realizado más de un pedido:

```
clientes = (pedidos.groupby("correo")
            .agg(num_pedidos=("id_pedido", "count"),
                 fecha_adquisicion=("fecha_pedido", "min"),
                 estado=("estado", "first"))
            .reset_index())

clientes["es_recurrente"] = clientes["num_pedidos"] > 1
clientes["clientes_mes_año"] = clientes["fecha_adquisicion"].dt.to_period("M").astype(str)
```

4.3 Importe de primer pedido e importe de pedidos recurrentes

Se separan los pedidos de cada cliente en "primer pedido" (`num_pedido_cliente == 1`) y "pedidos recurrentes" (`num_pedido_cliente > 1`), y se calcula el importe correspondiente a cada grupo, que después se incorpora a la tabla de clientes:

```
primeros = pedidos[pedidos["num_pedido_cliente"] == 1]
recurrentes = pedidos[pedidos["num_pedido_cliente"] > 1]

clientes = clientes.merge(
    primeros.rename(columns={"importe_pedido": "importe_primer_pedido"})
    [["correo", "importe_primer_pedido"]], on="correo", how="left")

importe_recurrente = (recurrentes.groupby("correo")["importe_pedido"].sum()
    .rename("importe_total_recurrente").reset_index())
clientes = clientes.merge(importe_recurrente, on="correo", how="left")
clientes["importe_total_recurrente"] = clientes["importe_total_recurrente"].fillna(0)
```

4.4 Tiempo medio entre recurrencias

Para cada cliente se calcula la diferencia en días entre cada pedido y el anterior. Se genera además una variante que excluye los pedidos realizados el mismo día (diferencia = 0 días), para evitar que estos distorsionen a la baja la media del tiempo entre recurrencias:

```
pedidos["dias_desde_anterior"] = pedidos.groupby("correo")["fecha_pedido"].diff().dt.days

pedidos["dias_desde_anterior_sin_sameday"] = pedidos["dias_desde_anterior"].where(
    pedidos["dias_desde_anterior"] > 0)

tiempo_medio = (pedidos.groupby("correo")
    .agg(
        tiempo_medio_recurrencia_dias=("dias_desde_anterior", "mean"),
        tiempo_medio_sin_sameday=("dias_desde_anterior_sin_sameday",
    "mean")
    )
    .reset_index())

clientes = clientes.merge(tiempo_medio, on="correo", how="left")
```

4.5 Exportación de resultados

Finalmente, se exportan a CSV las tres tablas resultantes, que son las que se cargan en Power BI para construir el cuadro de mando: el detalle limpio a nivel de línea de pedido, la tabla de pedidos y la tabla de clientes con las métricas ya calculadas.

```
df.to_csv("dataset_limpio.csv")
pedidos.to_csv("pedidos.csv")
clientes.to_csv("clientes.csv")
```

4.6 Resumen de métricas y su valor

Métrica	Definición	Resultado
% clientes recurrentes	Proporción de clientes recurrentes sobre el total de clientes, calculable por año/mes de adquisición o por estado.	47,81 %

Métrica	Definición	Resultado
Importe primer pedido	Importe del primer pedido de cada cliente; se analiza como promedio por segmento.	1.878,49 €
Importe pedidos recurrentes	Suma de los pedidos posteriores al primero; se analiza como promedio por segmento.	12.268,57 €
Tiempo medio entre recurrencias	Media de días transcurridos entre pedidos consecutivos del mismo cliente.	31,77 días

5. Cuadro de mando en Power BI

Con las tablas pedidos.csv y clientes.csv se ha construido en Power BI un cuadro de mando con dos vistas: una vista de Dashboard con indicadores y gráficos, y una vista de Tablas resumen con el detalle desglosado por año/mes y por estado.

5.1 Vista Dashboard

Esta vista muestra los indicadores clave del negocio (número de clientes, % de clientes recurrentes, tiempo medio entre pedidos, facturación total, importe medio del primer pedido e importe medio de los pedidos recurrentes), junto con el desglose de facturación por categoría de producto, la evolución mensual de la facturación, la facturación por género y un filtro interactivo por estado.

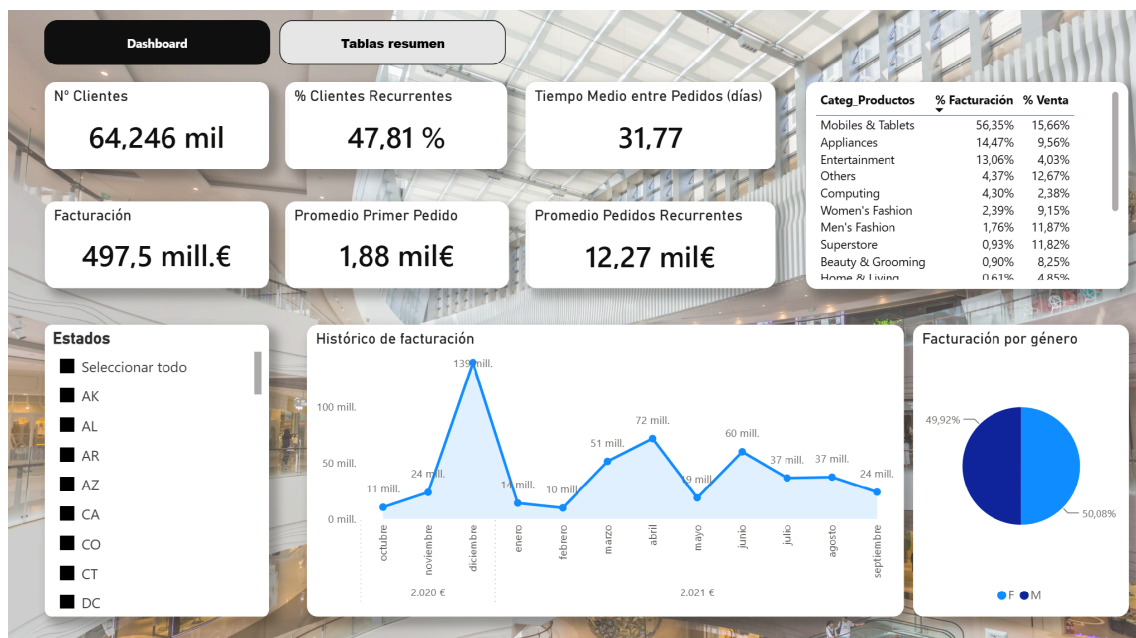


Figura 1. Vista Dashboard — indicadores clave, evolución mensual y desglose por categoría, género y estado.

5.2 Vista Tablas resumen

Esta vista complementa al dashboard con el detalle numérico de las métricas, desglosado por año y mes de adquisición del cliente (primera tabla) y por estado/provincia (segunda tabla), permitiendo comparar la evolución temporal frente a la dispersión geográfica.

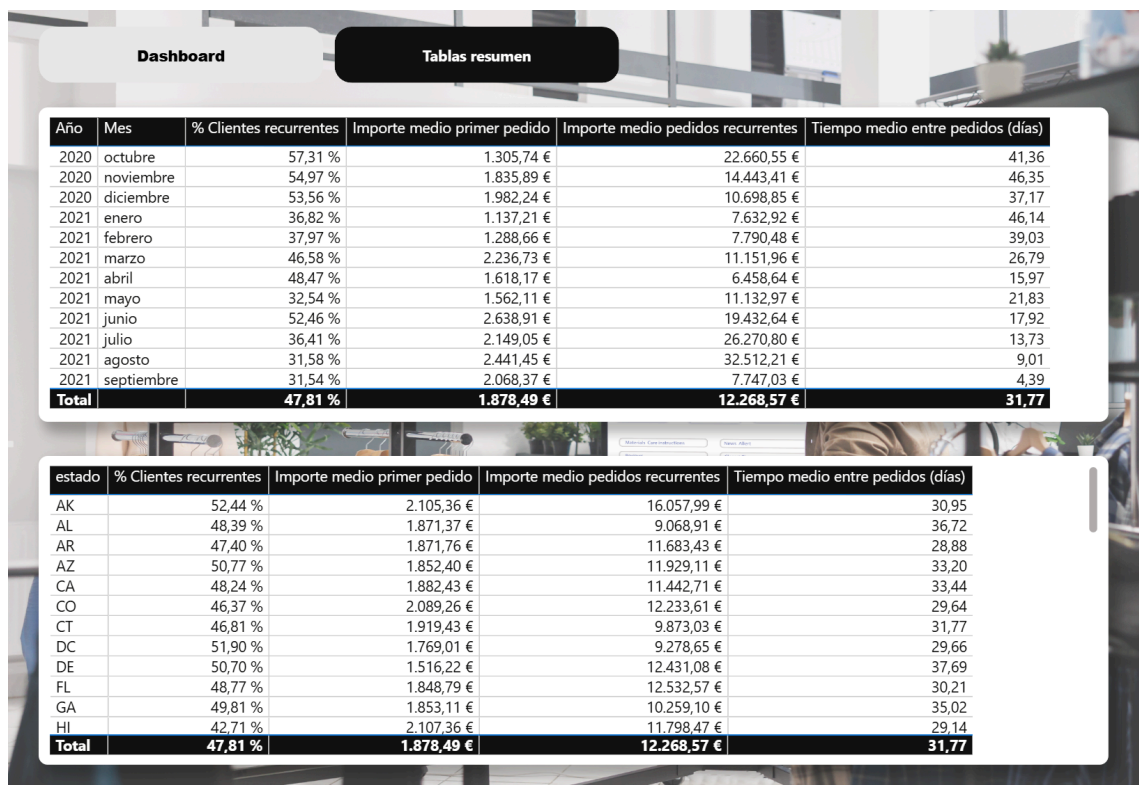


Figura 2. Vista Tablas resumen — métricas desglosadas por año/mes y por estado.

6. Estructura del repositorio

- dataset.json — dataset original de partida.
- script_Retail.ipynb — notebook con todo el proceso de limpieza, transformación y cálculo de métricas.
- dataset_limpio.csv — detalle a nivel de línea de pedido, ya limpio y tipado.
- pedidos.csv — tabla agregada a nivel de pedido.
- clientes.csv — tabla agregada a nivel de cliente, con las métricas de negocio calculadas.
- dashboard.pbix — cuadro de mando en Power BI (Dashboard + Tablas resumen).

7. Conclusiones

El proceso implementado permite pasar de un dataset semi-estructurado en JSON a un conjunto de tablas analíticas limpias y a un cuadro de mando que responde a los requisitos planteados: desglose por año/mes de adquisición y por estado, y cálculo de las cuatro métricas clave solicitadas (% de clientes recurrentes, importe de primeros pedidos, importe de pedidos recurrentes y tiempo medio entre recurrencias).

7.1 Recomendaciones de negocio

A partir de los resultados obtenidos se plantean las siguientes recomendaciones accionables para negocio:

- Priorizar la fidelización sobre la captación: el importe medio de los pedidos recurrentes (12.269 €) multiplica por más de 6 el del primer pedido (1.878 €). Un cliente que repite compra genera muchísimo más valor que uno nuevo, por lo que se recomienda destinar parte del presupuesto de marketing a incentivar la segunda compra (descuento de bienvenida a la recurrencia, envío gratuito...) en lugar de concentrarse sólo en adquisición.

- Diseñar campañas de reactivación en torno al día 30: el tiempo medio global entre pedidos recurrentes es de 31,77 días. Se recomienda automatizar comunicaciones (email/push) alrededor de los días 25–28 tras la última compra de cada cliente, antes de que la probabilidad de reactivación caiga.
- Vigilar la evolución de la recurrencia con cautela: la tabla por año/mes muestra un descenso del % de clientes recurrentes desde el 57,31 % (octubre 2020) hasta el 31,54 % (septiembre 2021). Parte de este descenso es un efecto esperado de la ventana de observación (los clientes captados más tarde han tenido menos tiempo para repetir compra), por lo que no debe interpretarse aún como una caída real de la fidelización; se recomienda volver a calcular esta métrica dentro de unos meses, cuando los clientes de 2021 hayan tenido el mismo tiempo de maduración que los de 2020, para confirmar si la tendencia se mantiene.
- Aprovechar el ticket de entrada creciente: el importe medio del primer pedido ha crecido de forma sostenida, de 1.305,74 € (octubre 2020) a 2.068,37 € (septiembre 2021). Conviene analizar si este mayor ticket inicial se traduce también en mayor recurrencia o si, por el contrario, los clientes de ticket alto compran una sola vez, para ajustar la estrategia de producto en la primera compra.
- Replicar buenas prácticas entre estados: la fidelización varía sensiblemente por estado, desde el 52,44 % de recurrencia en AK hasta el 42,71 % en HI. Se recomienda estudiar qué diferencia la experiencia de compra, la logística o el mix de producto en los estados con mejor recurrencia, para extender esas prácticas a los estados más rezagados.
- Reducir la dependencia de una única categoría: Mobiles & Tablets concentra el 56,35 % de la facturación total pero solo el 15,66 % del volumen de venta, lo que evidencia una fuerte dependencia de una categoría de ticket unitario alto. Se recomienda potenciar categorías con buen equilibrio entre volumen y fidelización (por ejemplo Women's o Men's Fashion, con alto % de venta) mediante cross-selling en el primer pedido, para diversificar el riesgo comercial.